

Error handling

For a large-scale distributed system like YouTube, failures are unavoidable. Hardware crashes, network failures, corrupted files, and software bugs can happen at any time.

To build a highly fault-tolerant system, errors must be:

- Detected quickly
- Handled gracefully
- Recovered automatically whenever possible

Types of errors

There are mainly two categories of errors:

1. Recoverable errors
2. Non-recoverable errors

Recoverable errors

Recoverable errors are temporary failures from which the system can recover automatically.

Examples:

- Temporary network failure
- Worker crash
- Video segment transcoding failure

Recovery strategy

The common solution is:

- Retry the failed operation multiple times

If retries succeed:

- Processing continues normally

If retries continuously fail:

- The system returns an appropriate error response to the client

Non-recoverable errors

Non-recoverable errors are permanent failures that cannot be fixed through retries.

Examples:

- Malformed video format
- Unsupported codec
- Corrupted uploaded file

Recovery strategy

The system:

- Stops all associated tasks
- Cleans temporary resources
- Returns proper error messages to users

This avoids wasting computation resources.

Error handling playbook

The system contains many distributed components. Each component requires specific recovery mechanisms.

Upload error

Cause

- Network interruption
- Packet loss
- Client disconnection

Solution

- Retry upload multiple times
- Use resumable chunk uploads

GOP-based chunking helps retry only failed chunks instead of entire videos.

Split video error

Some old clients may not support GOP-based splitting.

Solution

If client-side splitting fails:

- Upload entire video to server
- Server performs GOP splitting

This ensures backward compatibility.

Transcoding error

Video transcoding may fail because:

- Worker crashes
- Corrupted chunks
- Resource exhaustion

Solution

- Retry transcoding task
- Reassign task to another worker if needed

Since videos are split into chunks, only failed chunks require retranscoding.

Preprocessor error

The preprocessor may fail while:

- Splitting video
- Generating metadata
- Building DAG pipelines

Solution

- Regenerate DAG configuration
- Retry preprocessing operations

Cached GOP chunks help avoid complete reprocessing.

DAG scheduler error

The DAG scheduler may fail while:

- Scheduling tasks
- Creating execution stages

Solution

- Reschedule failed tasks
- Recover DAG state from persistent storage

This ensures workflow continuity.

Resource manager queue failure

The resource manager depends heavily on task queues.

Queue failures can halt the entire transcoding pipeline.

Solution

Use replicated queues.

If one queue instance fails:

- Traffic automatically switches to replica queues

Benefits:

- High availability
- Fault tolerance
- No single point of failure

Task worker failure

Workers may crash because of:

- Hardware failure
- Software bugs
- Memory exhaustion

Solution

- Detect worker heartbeat failure
- Reassign tasks to healthy workers
- Retry execution

The resource manager updates:

- Worker queue
- Running queue

to reflect worker status changes.

API server failure

API servers are stateless.

Thus, requests are not tied to specific servers.

Solution

If an API server fails:

- Load balancer redirects requests to another API server

Benefits:

- High availability
- Easy horizontal scaling

Metadata cache server failure

Metadata cache stores:

- Video metadata
- User objects
- Frequently accessed information

Solution

Data is replicated across multiple cache nodes.

If one cache server fails:

- Other replicas continue serving requests
- New cache node replaces failed node

Benefits:

- Continuous availability
- Fast recovery

Metadata database failure

Metadata databases are replicated using:

- Master-slave replication

Two failure scenarios exist.

Master database failure

If the master database fails:

- One slave is promoted as the new master

This process is called **failover**.

Benefits:

- Minimal downtime
- Continued write availability

Slave database failure

If a slave fails:

- Other slaves continue serving read requests
- New slave node is created to replace failed replica

Benefits:

- High read availability
- Improved fault tolerance

Importance of retries

Retries are one of the most important recovery mechanisms in distributed systems.

However, retries must be carefully controlled using:

- Retry limits
- Exponential backoff
- Timeout handling

Otherwise:

- Retry storms may overload the system

Importance of replication

Replication improves:

- Reliability
- Availability
- Disaster recovery

Critical components that require replication:

- Metadata DB
- Cache servers
- Queues
- Blob storage

Importance of monitoring

Error handling also requires continuous monitoring.

Monitoring systems track:

- Failed uploads
- Worker crashes
- Queue delays
- CDN failures
- API latency

Popular monitoring tools:

- Prometheus
- Grafana
- Datadog

Alerts help engineers respond quickly to failures.

Summary of error handling mechanisms

Component	Failure Type	Recovery Strategy
Upload service	Upload interruption	Retry upload
Video splitting	GOP split failure	Server-side splitting
Transcoding worker	Encoding failure	Retry task
Preprocessor	DAG generation failure	Regenerate DAG
DAG scheduler	Scheduling failure	Reschedule tasks
Resource manager queue	Queue failure	Use replicas
Task worker	Worker crash	Reassign tasks
API server	Server down	Redirect via load balancer

Metadata cache	Cache node failure	Use replicas
Metadata DB master	Master failure	Promote slave
Metadata DB slave	Replica failure	Replace slave

Key goals of error handling system

The error handling system is designed to achieve:

- High availability
- Fault tolerance
- Automatic recovery
- Minimal downtime
- Reliable video processing
- Smooth user experience